CSE 144 Final Project Report

Transfer Learning Challenge
Group Members: Sequoia Boubion-McKay, Kevin Chan, Caleb Cho

Spring 2026

1. Introduction

The goal of this project was to build a high-accuracy classifier for the UCSC CSE
144 Spring 2026 transfer-learning Kaggle challenge. The task is 100-way image classification from a small training set, with predictions submitted for an unlabeled test directory.

Transfer learning is the right fit for this setting because the available training data is too small and imbalanced to train a modern image model from scratch. The project therefore starts from strong ImageNet-pretrained backbones,
replaces the classifier with a 100-class head, and fine-tunes the full network with regularization and cross-validation.

The final pipeline uses `timm` ConvNeXt backbones pretrained on ImageNet-21k and
fine-tuned on the provided `Data/train/` images only. The best single offline model was ConvNeXt-B with 0.7961 out-of-fold accuracy. The best confirmed Kaggle
public result recorded for the project was 0.8000 accuracy from the ConvNeXt-S submission. An equal ConvNeXt-S + ConvNeXt-B OOF ensemble was tested and did not
improve over ConvNeXt-B alone.

2. Dataset

The dataset is stored under `Data/`:

```
Data/
  train/
    0/
    1/
    ...
    99/
  test/
    0.jpg
    1.jpg
    ...
  sample_submission.csv
```

The verified training set contains 1,079 images across 100 classes. Class folders
are named `0` through `99`, and the folder name is the ground-truth label. The test directory contains 1,036 images named `0.jpg` through `1035.jpg`.

The class distribution is imbalanced: the smallest class has 4 images, the largest has 41 images, and the mean is 10.79 images per class. The count

distribution is:

Images per class	Number of classes
4	1
5	4
6	4
7	3
8	7
9	2
10	33
11	28
12	5
13	3
14	4
15	1
16	1
20	2
31	1
41	1

A major correctness issue is label mapping. PyTorch `ImageFolder` would sort folder strings alphabetically as `0, 1, 10, 11, ...`, which would scramble labels.
The implementation avoids this by building an explicit mapping `label = int(folder_name)` and asserting that the labels are exactly `0..99`.

All images are opened with PIL and converted to RGB. Preprocessing uses per-backbone `timm` evaluation transforms, including the pretrained model's resolved input size, interpolation, crop percentage, mean, and standard deviation. The main models use 224 x 224 inputs.

Training augmentation uses:

• Random resized crop with minimum scale 0.4
• Horizontal flip with probability 0.5
• RandAugment `rand-m9-mstd0.5-inc1`
• Color jitter 0.4
• Random erasing probability 0.25
• MixUp with alpha 0.2
• CutMix with alpha 1.0

3. Implementation

3.1 Model

The main backbones are:

Alias	`timm` model	Input size	Role
`convnext_small`	`convnext_small.fb_in22k_ft_in1k`	224	Baseline submitted model
`convnext_base`	`convnext_base.fb_in22k_ft_in1k`	224	Strongest OOF single model

Both backbones use ImageNet-21k pretraining followed by ImageNet-1k fine-

tuning
before this project's fine-tuning step. This gives the model transferable visual
features before adapting to the 100 challenge classes.

For each model, the original classifier is replaced by a fresh 100-way classification head. Classifier dropout is set to 0.2, and stochastic depth uses
drop path 0.1. The final approach fine-tunes the full model rather than freezing
the backbone, but it uses layer-wise learning-rate decay so earlier pretrained layers update more conservatively than the classification head.

3.2 Training

The training script performs stratified 5-fold cross-validation. Each fold trains
on roughly 80% of the training images and validates on the held-out fold. The out-of-fold predictions are then assembled so every training image is evaluated
by a model that did not train on that image.

Core training settings:

Setting	Value
Seed	1337
Cross-validation	5-fold StratifiedKFold
Optimizer	AdamW
Peak head learning rate	1e-3
Minimum learning rate	1e-6
Weight decay	0.05
Layer-wise LR decay	0.8
Epochs	20 for current local runs
Warmup	4 epochs
Scheduler	Warmup + cosine decay
Gradient clipping	1.0
Label smoothing	0.1
EMA decay ceiling	0.9998 with warmup
AMP	Enabled on CUDA only

MixUp and CutMix use `timm.data.Mixup`, so the loss is `SoftTargetCrossEntropy`. When MixUp/CutMix are disabled for smoke tests, the code uses cross-entropy with label smoothing.

The code is device-agnostic and selects CUDA, then Apple MPS, then CPU. CUDA AMP
is used only on CUDA because this PyTorch/MPS stack had autocast issues. The local development environment was Apple M2 Pro / MPS with Python 3.11.5, PyTorch 2.1.2, torchvision 0.16.2, and `timm` 1.0.27. The recorded ConvNeXt-S Kaggle result came from a CUDA/Colab run; the ConvNeXt-B OOF result was produced
locally on MPS.

The implementation saves per-fold EMA checkpoints, OOF logits, and resolved

configuration snapshots. Checkpoint saves are atomic to avoid corrupting files if
a run is interrupted.

4. Experiments

Validation was based primarily on leak-free OOF accuracy rather than the Kaggle
public leaderboard. The public leaderboard covers only a portion of the test set,
so it was treated as a submission-format sanity check rather than a tuning target.

The main experiments were:

Experiment	Backbone(s)	OOF accuracy	Kaggle accuracy	Notes
Label-map smoke test	ConvNeXt-S, 10 classes	0.4500	N/A	Confirmed label wiring; random is 0.10
Baseline	ConvNeXt-S	0.7720 reported; 0.7785 local OOF file	0.8000	First strong result
Resolution ablation	ConvNeXt-S at 288, 15 epochs	0.7600	N/A	Did not improve baseline
Stronger backbone	ConvNeXt-B	0.7961	N/A	Best single OOF model
Equal ensemble	ConvNeXt-S + ConvNeXt-B	0.7924	N/A	Worse than ConvNeXt-B alone
Softer augmentation profile	ConvNeXt-B planned	N/A	N/A	Implemented but not used as final evidence

The ConvNeXt-S baseline initially achieved 0.7720 OOF accuracy with fold
accuracies around 0.80, 0.78, 0.77, 0.745, and 0.767. A later local OOF file for
ConvNeXt-S reports 0.7785. ConvNeXt-B improved to 0.7961 with per-fold best
validation accuracies:

Fold	ConvNeXt-B best validation accuracy
0	0.7917
1	0.8056
2	0.7963
3	0.7824
4	0.8047

The equal-average OOF ensemble over the two ConvNeXt models covered all 1,079
training samples but scored 0.7924, below ConvNeXt-B alone. This suggests the
two models were not diverse enough for a simple equal-weight ensemble to help.

5. Results

The best confirmed offline validation result was:

- Best OOF single model: ConvNeXt-B, 0.7961 accuracy
- Best confirmed Kaggle public accuracy: 0.8000 from ConvNeXt-S
- Equal ConvNeXt-S + ConvNeXt-B OOF ensemble: 0.7924

The Kaggle submission requirement was also corrected during the project.

Although
`sample_submission.csv` contains 1,000 rows, the actual test directory
contains
1,036 images. A 1,000-row submission was rejected, so `predict.py` now
enumerates
all test files numerically and validates that the output contains all 1,036
rows.

Error analysis from the OOF ensemble showed that the weakest classes were
mostly
tail or difficult classes. The lowest-recall classes in the equal ensemble
were:

Class	Recall
68	0.09
64	0.12
71	0.17
86	0.18
66	0.25
76	0.27
80	0.36
73	0.40
99	0.45
81	0.45

These failures are consistent with the small-data setting: some classes have
too
few examples to support robust fine-tuning, and visually similar classes are
more
likely to be confused.

6. Discussion

The strongest part of the approach was starting from ImageNet-21k pretrained
ConvNeXt models and fine-tuning carefully. ConvNeXt-S already cleared the
baseline comfortably, and ConvNeXt-B gave the best offline validation
accuracy.
Layer-wise LR decay, warmup/cosine scheduling, MixUp/CutMix, label smoothing,
dropout, stochastic depth, and EMA all address the main risk in this project:
overfitting a high-capacity model to roughly 11 images per class.

The most important engineering decision was the explicit numeric label map. A
silent label-order bug would have made the model appear to train while
producing
near-random Kaggle predictions. The submission validator was similarly
important
because the test folder has 1,036 images even though the sample submission has
1,000 rows.

Not every added component helped. Increasing resolution to 288 did not improve
OOF accuracy. The simple equal ensemble also failed to beat ConvNeXt-B alone,
which indicates that the two ConvNeXt models were making similar mistakes or
that
the weaker model diluted the stronger model's predictions.

The main limitation is the dataset size and imbalance. The weakest classes have
very low recall, and the model has little evidence for classes with only a few
training examples. Better next steps would be class-balanced sampling, rare-class
targeted augmentation, OOF-tuned ensemble weights, temperature tuning, and adding
a genuinely different architecture such as EfficientNetV2 or ViT only if it gives
a measured OOF lift.

7. Reproducibility

The project uses a fixed seed of 1337 through `seed_everything`, which seeds
Python, NumPy, and PyTorch and sets cuDNN deterministic behavior. Exact
bitwise-identical results are not guaranteed on GPU or MPS because some kernels
remain nondeterministic, but the data splits and run configuration are fixed.

Package versions are pinned in `requirements.txt`:

Package	Version
torch	2.1.2
torchvision	0.16.2
timm	1.0.27
scikit-learn	1.3.0
pandas	2.3.2
numpy	1.24.4
pyyaml	6.0.2
pillow	11.0.0

Install dependencies locally with:

```
pip install -r requirements.txt
```

Train ConvNeXt-B across all five folds:

```
PYTORCH_ENABLE_MPS_FALLBACK=1 python src/train.py \
  --backbone convnext_base \
  --device mps \
  --batch-size 8 \
  --out checkpoints_local
```

Train the ConvNeXt-S baseline:

```
python src/train.py --backbone convnext_small --out checkpoints_local
```

Report OOF ensemble accuracy:

```
python src/oof_ensemble.py \
  --ckpt-dir checkpoints_local \
  --backbones convnext_small,convnext_base
```

Generate a full 1,036-row submission:

```
python src/predict.py \
  --ckpt-dir checkpoints_local \
  --backbones convnext_base \
  --out outputs/submissions/submission.csv
```

The inference script validates the output columns, ID order, row count, and label
range before writing the CSV.

8. Team Contributions

Sequoia Boubion-McKay completed the model pipeline, report, experiment analysis,
and approach described in this document. This included the data-loading and label
mapping checks, ConvNeXt transfer-learning setup, training and inference scripts,
OOF validation workflow, experiment tracking, submission validation, and final writeup.

Kevin Chan and Caleb Cho worked on separate model approaches. As a group, we compared the available model results and shipped the best-performing solution for
the final submission.

9. References

• UCSC CSE 144 Spring 2026 Final Project handout. Local project copy:
  `Directions/md/CSE144_Project_Page.md`.
• PyTorch documentation, `torch.utils.data`.
  https://docs.pytorch.org/docs/2.12/data.html
• PyTorch documentation, "Reproducibility".
  https://docs.pytorch.org/docs/2.12/notes/randomness.html
• TorchVision documentation, "Models and pre-trained weights".
  https://docs.pytorch.org/vision/stable/models.html
• `timm` documentation, "Models" reference.
  https://huggingface.co/docs/timm/reference/models
• `timm` documentation, "Data" reference for `create_transform` and
  `resolve_data_config`. https://huggingface.co/docs/timm/reference/data
• Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell,
  and Saining Xie. "A ConvNet for the 2020s." arXiv:2201.03545.
  https://arxiv.org/abs/2201.03545
• Ilya Loshchilov and Frank Hutter. "Decoupled Weight Decay Regularization."
  arXiv:1711.05101. https://arxiv.org/abs/1711.05101